

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна

Студент гр. 4345

Гавриш М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм поиска расстояния Левенштейна.

Задание

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` $\rightarrow$ `stal`.
- Затем необходимо заменить два последних символа: `stal` $\rightarrow$ `stie`.
- Потом нужно добавить символ в конец строки: `stie` $\rightarrow$ `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Вариант 7б:

"Прóклятые элементы первой строки": на вход дополнительно подаётся список индексов 1-ой строки, элементы по которым запрещено заменять или удалять, но если прóклятым оказался символ "Z", то заменять его можно, нельзя только удалять.

Основные теоретические положения

Расстояние Левенштейна (редакционное расстояние) — это числовая характеристика, показывающая, насколько две строки отличаются друг от друга. Формально, это минимальное количество односимвольных операций, которые необходимо выполнить, чтобы преобразовать одну строку в другую.

Разрешённые операции:

Вставка — добавить один символ в произвольную позицию строки;

Удаление — убрать один символ из строки;

Замена — поменять один символ на другой.

Данная метрика находит применение в системах проверки орфографии, алгоритмах нечёткого поиска и других областях, где требуется оценить схожесть текстовых данных.

Для вычисления расстояния Левенштейна используется алгоритм Вагнера-Фишера, основанный на методе динамического программирования. Строится матрица D размером $(n+1) \times (m+1)$, где n и m — длины входных строк. Элемент $D[i][j]$ хранит минимальную стоимость преобразования префикса первой строки длины i в префикс второй строки длины j .

Заполнение матрицы происходит по рекуррентной формуле:

$$D[i][j] = \min(\begin{aligned} &D[i-1][j] + 1, && // \text{удаление символа из первой строки} \\ &D[i][j-1] + 1, && // \text{вставка символа из второй строки} \\ &D[i-1][j-1] + \text{cost} && // \text{замена или совпадение} \end{aligned})$$

где $\text{cost} = 0$, если символы совпадают ($S[i-1] == T[j-1]$), и $\text{cost} = 1$ в противном случае.

Базовые случаи:

$D[0][0] = 0$ — пустая строка уже равна пустой;

$D[i][0] = i$ — чтобы получить пустую строку из i символов, нужно удалить их все;

$D[0][j] = j$ — чтобы получить строку длины j из пустой, нужно вставить j символов.

Ответ находится в ячейке $D[n][m]$.

Выполнение работы

1. Оценка сложности алгоритма

Классическая реализация алгоритма Вагнера-Фишера требует:

Время: $O(n \times m)$ — необходимо заполнить каждую ячейку матрицы;

Память: $O(n \times m)$ — хранение всей матрицы размером $(n+1) \times (m+1)$.

2. В рамках варианта 7б в алгоритм внесены следующие изменения:

- Проклятые элементы: на вход дополнительно подаётся набор индексов первой строки. Символы на этих позициях имеют ограничения на операции.
- Запрет на удаление: если символ помечен как «проклятый», операция удаления для него становится недоступной независимо от его значения.
- Особое правило для символа 'z': если проклятый символ равен 'z', операция замены для него разрешена. Для всех остальных проклятых символов замена запрещена.
- Вставка всегда доступна: операция вставки не затрагивает символы первой строки, поэтому она разрешена в любом случае.

3. Описание программы:

- `const int INF = 1000000000;` — Константа, обозначающая «бесконечность» — значение, которое заведомо больше любого возможного ответа. Используется для маркировки недостижимых состояний в матрице.
- `vector<bool> is_cursed(n, false);` — Массив флагов, где `is_cursed[i] == true`, если символ `s[i]` является проклятым. Позволяет за $O(1)$ проверять ограничения для текущего символа.
- `vector<vector<int>> dp(n + 1, vector<int>(m + 1, INF));` — Двумерный вектор для хранения матрицы динамического программирования. Инициализируется значением `INF`, после чего заполняется согласно алгоритму.
- `bool can_replace = !is_cursed[i-1] || (s[i-1] == 'z');` — Замена разрешена, если символ не проклят ИЛИ если это проклятый, но равный 'z'.

- $dp[i][j] = \min(\{insert_cost, delete_cost, match_replace_cost\});$ — Стандартный шаг динамического программирования: из трёх возможных переходов выбирается тот, который даёт наименьшую суммарную стоимость.

Результаты тестирования программы см. в таблице 1. Исходный код программы см. в приложении А.

Таблица 1 – Тестирование программы

Ввод	Результат	Комментарий
pedestal stien	7	Пример из условия задачи
entrance reenterable	5	Пример из условия задачи
pedestal stien 2	9	Пример с проклятым символом e
pizza pasta 1 3	3	Проклятые символы: p, z
pizza pasta 2	-	Вычислить расстояние Левенштейна невозможно

Выводы

В ходе выполнения лабораторной работы:

1. Реализован алгоритм Вагнера-Фишера для вычисления расстояния Левенштейна.
2. Алгоритм модифицирован в соответствии с вариантом 7б: добавлена поддержка «проклятых» символов с особым правилом для 'z'.
3. Добавлен подробный вывод промежуточных шагов для демонстрации работы динамического программирования.
4. Проведено тестирование на различных наборах данных, подтвердившее корректность реализации.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

файл lb3_7b.cpp:

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

using namespace std;

const int INF = 1000000000;

string val(int x) {
    return (x >= INF) ? "inf" : to_string(x);
}

int main() {
    string s, t;
    cout << "Введите первую строку: ";
    cin >> s;
    cout << "Введите вторую строку: ";
    cin >> t;

    int n = s.length();
    int m = t.length();

    cout << "Введите номера проклятых элементов через пробел: ";
    vector<bool> is_cursed(n, false);
    string line;
    if (getline(cin >> ws, line)) {
        istringstream iss(line);
        int idx;
        while (iss >> idx) {
            if (idx >= 1 && idx <= n) {
                is_cursed[idx - 1] = true;
            }
        }
    }

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, INF));
    dp[0][0] = 0;

    for (int i = 1; i <= n; ++i) {
        if (!is_cursed[i-1]) {
            if (dp[i-1][0] != INF) dp[i][0] = dp[i-1][0] + 1;
        } else {
            dp[i][0] = INF; // Заблокировано
        }
        cout << "dp[" << i << "][0] = " << (dp[i][0] == INF ? "inf" :
to_string(dp[i][0])) << endl;
    }

    for (int j = 1; j <= m; ++j) {
        dp[0][j] = j;
        cout << "dp[0][" << j << "] = " << dp[0][j] << endl;
    }
}
```

```

    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            cout << "\nСимволы: " << s[i-1] << " из первой строки, "
                << t[j-1] << " из второй строки" << endl;

            if (is_cursed[i-1]) {
                cout << "Элемент " << s[i-1] << " проклят" << endl;
            }

            int insert_cost = INF, delete_cost = INF,
match_replace_cost = INF;

            if (dp[i][j-1] != INF) { // Insert
                insert_cost = dp[i][j-1] + 1;
                cout << "Стоимость вставки: " << insert_cost << endl;
                dp[i][j] = min(dp[i][j], dp[i][j-1] + 1);
            } else {
                cout << "Стоимость вставки: inf" << endl;
            }

            if (!is_cursed[i-1]) { // Delete
                if (dp[i-1][j] != INF) {
                    delete_cost = dp[i-1][j] + 1;
                    cout << "Стоимость удаления: " << delete_cost <<
endl;

                    dp[i][j] = min(dp[i][j], dp[i-1][j] + 1);
                } else {
                    cout << "Стоимость удаления: inf" << endl;
                }
            } else {
                cout << "Стоимость удаления: inf (проклятый символ)"
<< endl;
            }

            if (s[i-1] == t[j-1]) { // Match
                if (dp[i-1][j-1] != INF) {
                    match_replace_cost = dp[i-1][j-1];
                    cout << "Символы совпали, стоимость совпадения: "
<< match_replace_cost << endl;
                    dp[i][j] = min(dp[i][j], dp[i-1][j-1]);
                } else {
                    cout << "Символы совпали, стоимость совпадения:
inf" << endl;
                }
            } else {
                bool can_replace = !is_cursed[i-1] || (s[i-1] == 'z');
                if (can_replace) { // Replace
                    if (dp[i-1][j-1] != INF) {
                        match_replace_cost = dp[i-1][j-1] + 1;
                        cout << "Символы не совпали, стоимость замены:
" << match_replace_cost << endl;
                        dp[i][j] = min(dp[i][j], dp[i-1][j-1] + 1);
                    } else {
                        cout << "Символы не совпали, стоимость замены:
inf" << endl;
                    }
                }
            }
        }
    }
}

```

```

        } else {
            cout << "Символы не совпали, замена запрещена
(проклятый символ не 'z')\" << endl;
        }
    }

    cout << "Заносим минимальное значение \" << (dp[i][j] ==
INF ? \"inf\" : to_string(dp[i][j]))
        << \" в текущую ячейку матрицы\" << endl;
    }
}

cout << \"\\nИтоговая матрица:\\n\";
for (int i = 0; i <= n; ++i) {
    for (int j = 0; j <= m; ++j) {
        if (dp[i][j] == INF) {
            cout << \"inf \";
        } else {
            cout << dp[i][j] << \" \";
        }
    }
    cout << endl;
}

if (dp[n][m] >= INF) {
    cout << \"Вычислить расстояние Левенштейна невозможно\" << endl;
} else {
    cout << \"Расстояние Левенштейна: \" << dp[n][m] << endl;
}

return 0;
}

```